

**ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA**
Universidad de Córdoba



TRABAJO FIN DE GRADO

Grado en Ingeniería Informática
Mención en Ingeniería del Software

Sistema de Visualización de Métricas para Equipos Ágiles con Scrum/Kanban

(Manual técnico, manual de usuario y manual de código)

Autor

Ángel Eduardo Bauste De Nicolais

Director(es)

María Luque Rodríguez
Francisco Gracia Ahufinger

Agosto, 2026



Declaración de originalidad del TFG

Ángel Eduardo Bauste De Nicolais con DNI nº 03527704X, estudiante del Grado en Ingeniería Informática,

DECLARA

Que es el autor del presente trabajo siendo fruto de su propio esfuerzo y no constituye una reproducción total o parcial de otro trabajo. Todas las ideas, datos o fragmentos de otras obras han sido claramente señaladas, citadas y referenciadas conforme a las normas académicas y éticas.

Sistema de Visualización de Métricas para Equipos Ágiles con Scrum/Kanban

Resumen

Este Trabajo Fin de Grado propone el desarrollo de un sistema web de visualización de métricas para equipos ágiles que trabajan con Scrum y Kanban. La herramienta permitirá centralizar, analizar y mostrar indicadores clave como Velocity, Cycle Time, Lead Time y WIP, con el objetivo de mejorar la toma de decisiones basada en datos y favorecer la mejora continua.

El proyecto surge ante la dificultad que presentan muchos equipos para obtener una visión unificada de sus métricas, debido a la dispersión de la información entre distintas herramientas y al elevado coste de algunas soluciones comerciales. Para dar respuesta a esta necesidad, se diseñará una plataforma personalizable que obtendrá sus datos principalmente de Jira e integrará dashboards intuitivos, de modo que los equipos puedan tomar decisiones informadas y evaluar con transparencia el rendimiento del proyecto.

Palabras clave: Métricas ágiles, Scrum, Kanban, dashboard, visualización de datos, mejora continua, Jira, analítica de software.

Agile Team Metrics Visualization System with Scrum/Kanban

Abstract

This Bachelor's Thesis proposes the development of a web-based metrics visualization system for agile teams working with Scrum and Kanban. The tool will enable the centralization, analysis, and display of key indicators such as Velocity, Cycle Time, Lead Time, and WIP, with the aim of improving data-driven decision-making and supporting continuous improvement.

The project addresses the common difficulty many teams face when trying to obtain a unified view of their metrics, due to data being scattered across different tools and the high cost of existing commercial solutions. To address this need, a customizable platform will be designed to obtain its data mainly from Jira and integrate intuitive dashboards, allowing teams to make informed decisions and assess project performance transparently.

Keywords: Agile metrics, Scrum, Kanban, dashboard, data visualization, continuous improvement, Jira, software analytics.

Agradecimientos

Me gustaría dedicar unas palabras de agradecimiento a las personas que han ayudado a la consecución de este proyecto. En primer lugar, ...

Índice general

Índice general	IX
Índice de figuras	XI
Índice de tablas	XIII
1 Introducción	1
1.1 Motivación	2
1.2 Objetivos generales	2
1.3 Estructura de la memoria	3
2 Definición del problema	5
2.1 Contexto de los equipos ágiles	5
2.2 Situación actual y fragmentación de la información	6
2.3 Dificultad para transformar datos en métricas útiles	6
2.4 Limitaciones de las soluciones existentes	7
2.5 Necesidades detectadas	8
2.6 Usuarios y entorno de uso	8
2.7 Restricciones y condicionantes del problema	9
2.8 Alcance del problema	9
3 Requisitos	11
3.1 Introducción	11
3.2 Descripción general del sistema	11
3.3 Actores del sistema	12
3.4 Restricciones, supuestos y dependencias	12
3.5 Priorización de requisitos	12
3.6 Requisitos funcionales	13
3.6.1 Integración y sincronización con Jira	13
3.6.2 Configuración del análisis	13
3.6.3 Cálculo de métricas	14
3.6.4 Dashboard y visualización	15
3.6.5 Alertas y logs	15
3.6.6 Autenticación y control de acceso	16
3.7 Requisitos no funcionales	17

3.8	Requisitos de información	18
3.9	Casos de uso	20
3.10	Trazabilidad entre requisitos y casos de uso	21
Bibliografía		22
A Resultados detallados de la experimentación		25
	Referencias	25
B Manual de usuario		29
	Referencias	29
C Manual de código fuente		31
C.1	Contenidos del manual de código fuente	31
C.2	Código fuente de otros autores	32
C.3	¿Qué va a mirar el tribunal en tú código?	32
C.4	Estilo del texto de los ficheros de código fuente	33
	Referencias	35

Índice de figuras

A.1	Pantalla de acceso.	27
A.2	Pantalla de perfil de usuario.	27

Índice de tablas

3.10	Matriz de trazabilidad entre requisitos funcionales y casos de uso . . .	21
A.1	Resultados de regresión logística en Buzzfeed	26
A.2	CU01: Registro en la aplicación	28

Capítulo 1

Introducción

En la industria actual del desarrollo de software, la gestión de proyectos se apoya con frecuencia en enfoques ágiles, implementados habitualmente mediante marcos de trabajo como Scrum y Kanban. En este contexto, las métricas se convierten en una herramienta fundamental para que los equipos puedan medir su rendimiento, evaluar la eficiencia de su flujo de trabajo y diagnosticar la salud general del proceso de desarrollo. Indicadores como la velocidad de entrega (Velocity), el tiempo de ciclo (Cycle Time), el tiempo de entrega (Lead Time) y el trabajo en curso (WIP) permiten tomar decisiones basadas en evidencia y avanzar hacia la mejora continua.

Sin embargo, a pesar de la importancia de estas métricas, muchos equipos encuentran dificultades para obtener una visión unificada y útil de las mismas. La información relativa al trabajo suele estar fragmentada en distintas herramientas (por ejemplo, Jira para la gestión de tareas, GitHub para el código o plataformas de despliegue), y las soluciones comerciales que prometen integrar y visualizar estos datos a menudo tienen costes elevados o están orientadas a grandes organizaciones. Esta falta de visibilidad lleva a que, en la práctica, muchas decisiones se tomen más por intuición que por datos objetivos, lo cual limita la capacidad de mejorar el proceso de forma sistemática.

Este Trabajo Fin de Grado surge como respuesta a esta problemática, con el objetivo de desarrollar un sistema web de visualización de métricas para equipos ágiles que trabajan con Scrum y Kanban. La herramienta permitirá centralizar, analizar y mostrar indicadores clave a partir de los datos almacenados en Jira, fuente principal del sistema, de modo que los equipos puedan contar con dashboards intuitivos y personalizados que faciliten la toma de decisiones basada en datos y la evaluación transparente del rendimiento del proyecto.

1.1. Motivación

La motivación principal de este proyecto es democratizar el acceso a la analítica ágil de calidad, ofreciendo una solución de bajo coste, personalizable y replicable, que pueda ser utilizada tanto por equipos profesionales como por estudiantes o grupos de trabajo con recursos limitados.

Actualmente, herramientas como Jira permiten gestionar el trabajo de forma eficiente, pero sus capacidades analíticas avanzadas suelen estar disponibles solo en planes de pago o requieren la integración con soluciones externas de terceros. Además, estas soluciones tienden a ofrecer dashboards genéricos, con poca flexibilidad para adaptarse a las necesidades específicas de cada equipo. Esto genera una barrera importante para equipos pequeños o medianos, que no pueden asumir el coste de licencias premium pero que, al mismo tiempo, necesitan visibilidad sobre sus métricas para mejorar su forma de trabajar.

El presente TFG busca cubrir ese hueco mediante una plataforma que:

- Centralice los datos procedentes de Jira, evitando la dispersión de la información.
- Calcule automáticamente métricas clave como Velocity, Cycle Time, Lead Time y WIP.
- Ofrezca dashboards intuitivos y personalizables, adaptados a las necesidades de cada equipo.
- Sea técnica y económicamente replicable, utilizando tecnologías modernas y de amplio uso en la industria.

De esta forma, se pretende devolver a los equipos el control sobre sus procesos de mejora continua, basándose en evidencia y no solo en intuición.

1.2. Objetivos generales

El objetivo principal de este Trabajo Fin de Grado es construir un dashboard web funcional que permita a los equipos ágiles visualizar, analizar y gestionar sus métricas clave de Scrum y Kanban mediante paneles intuitivos y personalizables.

Para alcanzar este objetivo general, se plantean los siguientes subobjetivos:

- Implementar un módulo de captura de datos que se integre con Jira Cloud API para automatizar la obtención de información relevante de proyectos, tableros, sprints e incidencias.

- Diseñar e implementar dashboards personalizables que permitan a cada equipo seleccionar y visualizar sus métricas ágiles más relevantes mediante una interfaz responsive.
- Desarrollar un sistema de alertas que notifique al equipo cuando se detecten anomalías o tendencias preocupantes en las métricas.
- Validar la usabilidad de la plataforma mediante pruebas con equipos ágiles o simulaciones, con el objetivo de alcanzar un nivel de satisfacción del usuario elevado.
- Implementar las medidas de seguridad y privacidad necesarias, incluyendo autenticación robusta a través de Jira/Atlassian y protección de información sensible.

1.3. Estructura de la memoria

Esta memoria se organiza en varios capítulos y anexos que recogen de forma progresiva el análisis, diseño, desarrollo y validación del sistema:

1. La presente introducción, donde se contextualiza el proyecto y se resumen su motivación y objetivos principales.
2. La definición del problema, dedicada a describir la situación actual de los equipos ágiles y las limitaciones que justifican el desarrollo del sistema.
3. El capítulo de requisitos, donde se especifican las funcionalidades, restricciones y necesidades de información que debe cubrir la solución.
4. Los capítulos técnicos de diseño, implementación y validación, que se completarán conforme avance el desarrollo del sistema.
5. Los anexos, que incluirán el manual de usuario, el manual de código y cualquier material complementario necesario para comprender, desplegar o mantener la aplicación.

Capítulo 2

Definición del problema

El presente capítulo describe el problema que motiva el desarrollo del Trabajo Fin de Grado. Su objetivo no es presentar todavía la solución propuesta, sino analizar el contexto en el que aparece la necesidad, las limitaciones de la situación actual y los condicionantes que deben tenerse en cuenta antes de definir los requisitos del sistema.

2.1. Contexto de los equipos ágiles

En el desarrollo de software actual es habitual que los equipos organicen su trabajo mediante enfoques ágiles. Marcos como Scrum y Kanban proporcionan formas de planificar, visualizar y adaptar el trabajo a medida que avanza el proyecto. Scrum estructura el desarrollo en iteraciones temporales y promueve la inspección frecuente del producto y del proceso [1]. Kanban, por su parte, se centra en la visualización del flujo de trabajo, la limitación del trabajo en curso y la mejora evolutiva del sistema [2].

En ambos casos, la actividad diaria del equipo genera una gran cantidad de datos: incidencias, tareas, cambios de estado, sprints, tiempos de entrega, bloqueos, prioridades o información sobre el avance del trabajo. Estos datos pueden aportar valor si se transforman en métricas comprensibles, pero por sí solos no garantizan una mejora del proceso. De hecho, la adopción de prácticas ágiles no implica necesariamente que el equipo disponga de una visión clara de su rendimiento. El informe *17th State of Agile* de Digital.ai indica que el uso de Agile está muy extendido en el ciclo de vida del desarrollo software, pero también refleja que la satisfacción con su funcionamiento y su escalado no siempre es elevada [3].

Por tanto, el problema se sitúa en un entorno en el que los equipos sí cuentan con herramientas y datos, pero no siempre con mecanismos accesibles para convertir esa información en conocimiento útil para la toma de decisiones.

2.2. Situación actual y fragmentación de la información

Una parte importante de los equipos ágiles utiliza herramientas especializadas para gestionar su trabajo. Jira, por ejemplo, permite modelar proyectos, incidencias, tableros, sprints y flujos de trabajo, y se ha consolidado como una de las herramientas más utilizadas en este ámbito [4]. Sin embargo, la información relevante para evaluar el funcionamiento de un equipo no siempre se encuentra en una única plataforma.

En un proyecto software real, Jira puede contener la planificación y el estado de las tareas, mientras que repositorios como GitHub almacenan commits, ramas y solicitudes de integración. A su vez, las plataformas de integración y despliegue continuo pueden registrar información sobre builds, despliegues o fallos en producción. Esta separación no es necesariamente negativa, ya que cada herramienta cumple una función concreta. El problema aparece cuando el equipo necesita relacionar datos procedentes de distintas fuentes para entender cómo fluye realmente el trabajo.

En este contexto, la información puede quedar dispersa y requerir procesos manuales para ser recopilada, filtrada o interpretada. Esto dificulta obtener una visión histórica y coherente del proceso, especialmente en equipos pequeños, proyectos académicos o entornos donde no se dispone de herramientas avanzadas de analítica.

2.3. Dificultad para transformar datos en métricas útiles

El problema no consiste únicamente en almacenar datos del proyecto. Para que estos datos sean útiles deben ser seleccionados, relacionados y transformados en métricas que respondan a preguntas concretas del equipo. Por ejemplo, no basta con conocer cuántas incidencias hay en un tablero; puede ser necesario saber cuántas tareas se completan por sprint, cuánto tiempo tarda una tarea desde que se solicita hasta que se entrega, cuánto trabajo está simultáneamente en curso o en qué estados se producen los principales cuellos de botella.

Entre las métricas habituales en equipos Scrum y Kanban se encuentran la Velocity, el WIP, el Cycle Time y el Lead Time. La Velocity permite analizar la cantidad de trabajo completado en un sprint; el WIP ayuda a observar el trabajo en curso; el Cycle Time permite estudiar el tiempo que una tarea permanece en flujo activo; y el Lead Time ofrece una visión más amplia del tiempo total desde la solicitud hasta la entrega. Estas métricas pueden apoyar la mejora continua, pero requieren una definición consistente para evitar interpretaciones erróneas.

Además, algunas métricas procedentes del ámbito DevOps, como Deployment Frequency o Change Lead Time, están relacionadas con el rendimiento de entrega de software. La investigación de DORA las presenta como indicadores útiles para evaluar la capacidad de entregar software de forma rápida y estable [5]. No obstante, estas métricas no deben confundirse con las métricas ágiles de planificación y flujo, ni utilizarse fuera de contexto.

DORA advierte que convertir las métricas en objetivos aislados, comparar equipos con contextos distintos o medir sin orientar la mejora puede producir efectos contraproducentes [5].

Por ello, una necesidad central del problema es facilitar el cálculo y la interpretación de métricas sin perder de vista que su finalidad debe ser mejorar el sistema de trabajo, no vigilar individualmente a las personas ni generar rankings simplistas.

2.4. Limitaciones de las soluciones existentes

El mercado ya ofrece soluciones relacionadas con la gestión ágil y la analítica de equipos de desarrollo. Esta existencia confirma que el problema tiene relevancia práctica, pero también muestra que no todas las soluciones se ajustan al alcance, coste o nivel de personalización requerido por equipos pequeños o proyectos académicos.

Jira proporciona funcionalidades de gestión y generación de informes, y sus planes comerciales diferencian capacidades según el tipo de suscripción. En particular, Atlassian Analytics y Atlassian Data Lake aparecen asociados al plan Enterprise, lo que muestra que la analítica avanzada e integrada forma parte de niveles orientados a organizaciones de mayor tamaño [6]. Esto no significa que Jira carezca de métricas, sino que puede existir margen para desarrollar una capa especializada, controlable y adaptada al alcance concreto de este TFG.

Otras plataformas, como LinearB o Swarmia, integran datos procedentes de repositorios, sistemas de gestión de trabajo y herramientas de entrega para proporcionar métricas de ingeniería, visibilidad del flujo y análisis de cuellos de botella [7, 8]. Estas herramientas son referencias relevantes porque muestran la utilidad de combinar información de distintas fuentes. Sin embargo, su amplitud funcional y orientación empresarial exceden el objetivo académico de este proyecto.

También existen herramientas más ligeras de organización visual, como Trello o Tai-ga, que resultan útiles para representar tableros y coordinar tareas [9, 10]. Su principal valor está en la simplicidad, aunque no siempre ofrecen la profundidad analítica necesaria para estudiar métricas de flujo con detalle histórico. Por otro lado, plataformas integrales como Azure DevOps agrupan gestión de trabajo, repositorios y procesos de entrega, pero pueden resultar más amplias de lo necesario cuando el objetivo principal es comprender un conjunto concreto de métricas Scrum/Kanban [11].

En consecuencia, la oportunidad del TFG no consiste en reemplazar estas soluciones, sino en abordar un problema acotado: disponer de una herramienta especializada, replicable y de bajo coste que permita analizar métricas ágiles a partir de Jira como fuente principal de datos.

2.5. Necesidades detectadas

A partir del contexto anterior se identifican varias necesidades que el sistema deberá considerar posteriormente en la especificación de requisitos:

- Centralizar los datos relevantes procedentes de Jira para evitar consultas manuales repetitivas.
- Permitir definir el ámbito de análisis, por ejemplo mediante proyectos, tableros, sprints, etiquetas o tipos de incidencia.
- Calcular métricas ágiles de forma consistente, especialmente Velocity, WIP, Cycle Time y Lead Time.
- Presentar las métricas mediante dashboards claros, comprensibles y adaptables a distintos equipos.
- Facilitar el análisis histórico para observar tendencias y detectar posibles cuellos de botella.
- Mantener una solución técnicamente replicable, basada en tecnologías de uso común y sin dependencia obligatoria de licencias comerciales avanzadas.
- Proteger credenciales, tokens y datos sensibles asociados a la integración con Jira.
- Documentar el sistema de forma que pueda ser entendido, desplegado y mantenido por otros estudiantes o equipos pequeños.

Estas necesidades permiten enlazar la definición del problema con el capítulo de requisitos, donde se concretarán los servicios, restricciones y datos que debe manejar el sistema.

2.6. Usuarios y entorno de uso

El sistema se orienta principalmente a equipos que trabajan con enfoques ágiles y que utilizan Jira como herramienta de gestión. Dentro de este entorno pueden identificarse distintos perfiles de usuario. Un equipo de desarrollo puede utilizar las métricas para revisar la evolución de su flujo de trabajo; una persona con rol de Scrum Master o responsable de proceso puede emplearlas para preparar retrospectivas o detectar bloqueos; y un Product Owner o responsable de proyecto puede consultar la evolución del trabajo completado y la previsibilidad del equipo.

También se contempla un uso académico o de equipos pequeños. En este caso, el sistema puede servir como herramienta de aprendizaje y experimentación, permitiendo comprender cómo se calculan las métricas y cómo se relacionan con el funcionamiento

real de un proyecto. Este enfoque es importante porque el TFG no pretende construir una plataforma empresarial completa, sino una solución funcional y evaluable dentro de un alcance razonable.

2.7. Restricciones y condicionantes del problema

El problema presenta varias restricciones que condicionan la futura solución. La primera es que Jira Cloud API se toma como fuente principal de datos. Esto permite acotar la integración inicial y evitar que el sistema dependa desde el principio de múltiples plataformas externas. Aunque otras fuentes como GitHub o sistemas de CI/CD pueden aportar información adicional, su integración no debe considerarse imprescindible para una primera versión funcional.

Otra restricción relevante es la seguridad. La conexión con Jira puede requerir credenciales, tokens o mecanismos de autenticación que deben tratarse con especial cuidado. Por tanto, el sistema deberá evitar la exposición de información sensible y aplicar buenas prácticas de configuración y almacenamiento.

También deben considerarse los límites y condiciones de uso de las APIs externas. Un sistema que consulte datos de Jira de forma indiscriminada podría ser ineficiente o alcanzar límites de uso. Por ello, el problema exige pensar en mecanismos de sincronización, almacenamiento temporal o actualización bajo demanda que reduzcan peticiones innecesarias.

Finalmente, el alcance académico impone una restricción natural: el proyecto debe ser suficientemente completo para demostrar competencias de Ingeniería Informática, pero no puede pretender reproducir todas las funcionalidades de plataformas comerciales consolidadas. La solución deberá centrarse en un conjunto de métricas, integraciones y visualizaciones que puedan diseñarse, implementarse, validarse y documentarse de forma realista.

2.8. Alcance del problema

El problema que aborda este TFG se limita a la obtención, cálculo y visualización de métricas ágiles para equipos que gestionan su trabajo principalmente mediante Jira. Quedan dentro del alcance el análisis de datos de proyectos, tableros, sprints e incidencias; el cálculo de métricas como Velocity, WIP, Cycle Time y Lead Time; y la presentación de resultados mediante dashboards que ayuden a interpretar el estado y evolución del trabajo.

Queda fuera del alcance inicial sustituir a Jira como herramienta de gestión, reproducir una plataforma comercial completa de ingeniería, evaluar individualmente el rendimiento de desarrolladores o construir un sistema de analítica empresarial a gran escala. Esta delimitación permite enfocar el proyecto en una necesidad concreta y abordable, manteniendo coherencia con la naturaleza de un Trabajo Fin de Grado.

Capítulo 3

Requisitos

3.1. Introducción

En este capítulo se definen los requisitos que debe cumplir el sistema propuesto, estableciendo las funcionalidades principales, las restricciones técnicas y los datos necesarios para su funcionamiento. Esta especificación sirve como base para el diseño, la implementación y la posterior validación del sistema.

Para su elaboración se toma como referencia la práctica recomendada IEEE Std 830-1998, que propone que una especificación de requisitos software sea correcta, no ambigua, completa, consistente, verificable, modificable y trazable [12]. En este trabajo se adopta dicha referencia como guía para estructurar los requisitos y facilitar su posterior comprobación.

Asimismo, los requisitos se priorizan mediante la técnica MoSCoW, que clasifica cada requisito como *Must Have*, *Should Have*, *Could Have* o *Won't Have*. Esta técnica permite distinguir los requisitos imprescindibles para una versión funcional del sistema de aquellos que aportan valor, pero pueden planificarse como ampliaciones o mejoras [13].

3.2. Descripción general del sistema

El sistema consiste en una aplicación web orientada a la visualización de métricas ágiles para equipos que trabajan con marcos como Scrum y Kanban. Su objetivo principal es integrarse con Jira Cloud, obtener información de proyectos e incidencias, procesarla y mostrarla mediante un dashboard que facilite el análisis del flujo de trabajo, la evolución del equipo y determinados indicadores relacionados con la entrega de software.

La aplicación no pretende sustituir a Jira como herramienta de gestión del trabajo, sino proporcionar una capa complementaria de análisis centrada en métricas ágiles. De esta forma, los usuarios pueden consultar información agregada, aplicar filtros, revisar alertas y detectar posibles desviaciones en el comportamiento del equipo o del flujo de trabajo.

3.3. Actores del sistema

Se identifican los siguientes actores principales:

- **Usuario autenticado:** persona que accede al sistema mediante su cuenta de Atlassian y consulta dashboards, métricas, filtros, alertas y datos procedentes de Jira.
- **Jira Cloud:** sistema externo del que se obtienen los proyectos, tableros, sprints, incidencias, estados, fechas, estimaciones y demás datos necesarios para calcular las métricas.
- **Administrador o perfil técnico:** persona encargada de configurar aspectos técnicos de la aplicación, supervisar logs y mantener el correcto funcionamiento del sistema.

3.4. Restricciones, supuestos y dependencias

El sistema depende de la disponibilidad de Jira Cloud y de su API oficial para obtener los datos necesarios. Por tanto, cualquier limitación, cambio o indisponibilidad de dicha API puede afectar al funcionamiento de la aplicación.

Se asume que los proyectos analizados contienen información suficiente y correctamente mantenida para calcular las métricas previstas. Por ejemplo, fechas de creación y resolución, historial de cambios de estado, estimaciones, asignaciones, sprints o campos equivalentes según la configuración del proyecto.

También se considera que la autenticación se realizará mediante los mecanismos oficiales de Atlassian, evitando el registro local de usuarios y el uso de credenciales propias ajenas a Jira. Durante el desarrollo pueden emplearse configuraciones locales de prueba, pero el objetivo del sistema final es apoyarse en un flujo de autenticación adecuado para un entorno real.

3.5. Priorización de requisitos

La priorización de requisitos se realiza mediante la técnica MoSCoW. En este trabajo se interpreta cada categoría de la siguiente forma:

- **Must Have:** requisito imprescindible para que el sistema cumpla su propósito principal.
- **Should Have:** requisito importante, pero no estrictamente necesario para una primera versión funcional.

- **Could Have:** mejora deseable que aporta valor, pero que puede posponerse sin comprometer el objetivo central del sistema.
- **Won't Have:** requisito descartado para el alcance actual del TFG, aunque podría considerarse en trabajos futuros.

3.6. Requisitos funcionales

Los requisitos funcionales describen los servicios y funcionalidades que debe proporcionar el sistema. Para facilitar su lectura, se agrupan según el área funcional a la que pertenecen.

3.6.1. Integración y sincronización con Jira

ID	Prioridad	Descripción	Verificación
RF-01	Must	El sistema debe permitir conectar con Jira Cloud para importar los datos necesarios para calcular métricas y alimentar las vistas del sistema, incluyendo proyectos, tableros, sprints e incidencias.	Prueba de integración con Jira Cloud API.
RF-02	Must	El sistema debe permitir definir el alcance de la importación mediante filtros por proyecto, tablero, sprint, etiquetas, tipo de incidencia y otros criterios disponibles en Jira Cloud API que se incorporen al sistema.	Prueba funcional de filtros de importación.
RF-03	Must	El sistema debe permitir ejecutar procesos de sincronización de datos bajo demanda.	Prueba funcional de sincronización manual.
RF-04	Should	El sistema debería permitir programar sincronizaciones periódicas.	Prueba funcional de planificación de sincronizaciones.
RF-05	Must	El sistema debe registrar la fecha y hora de la última sincronización realizada para cada origen o ámbito de datos sincronizado.	Revisión de datos de sincronización almacenados.
RF-06	Should	El sistema debería detectar y señalar incidencias con datos incompletos o inconsistentes que puedan afectar al cálculo de métricas.	Prueba con incidencias de datos incompletos.

3.6.2. Configuración del análisis

ID	Prioridad	Descripción	Verificación
----	-----------	-------------	--------------

RF-07	Must	El sistema debe permitir configurar qué estados del flujo de trabajo representan el inicio y el fin del trabajo efectivo para el cálculo de métricas temporales.	Prueba funcional de configuración de estados.
RF-08	Should	El sistema debe permitir ajustar parámetros analíticos necesarios para adaptar los cálculos a distintos equipos o tableros, como estados de inicio y fin, rango temporal, campo de esfuerzo, percentiles utilizados y criterios de agrupación.	Prueba funcional de parámetros de análisis.
RF-18	Should	El sistema debería permitir registrar manualmente eventos necesarios para completar métricas cuando dichos datos no estén disponibles automáticamente.	Prueba funcional de registro manual de eventos.

3.6.3. Cálculo de métricas

ID	Prioridad	Descripción	Verificación
RF-09	Must	El sistema debe calcular automáticamente, como mínimo, las métricas Velocity, WIP, Lead Time y Cycle Time a partir de los datos sincronizados.	Prueba con conjunto de datos conocido.
RF-10	Should	El sistema debería calcular la métrica Deployment Frequency cuando disponga de datos suficientes para ello, por ejemplo mediante una integración con herramientas de control de versiones o CI/CD.	Prueba condicionada a la integración externa disponible.
RF-11	Must	El sistema debe recalcular las métricas afectadas tras cada sincronización completada con éxito.	Prueba de sincronización y recálculo posterior.
RF-12	Must	El sistema debe calcular Velocity por sprint a partir de las incidencias completadas y su esfuerzo asociado.	Prueba con datos de sprint y esfuerzo conocidos.
RF-13	Must	El sistema debe calcular el WIP actual y su evolución histórica por estado, columna, equipo, proyecto o periodo, según proceda.	Prueba con histórico de incidencias por estado.
RF-14	Must	El sistema debe calcular Lead Time y Cycle Time por incidencia y de forma agregada para periodos de análisis.	Prueba con transiciones temporales conocidas.
RF-15	Should	El sistema debería proporcionar estadísticas agregadas sobre las métricas temporales, incluyendo al menos media y percentiles.	Prueba de cálculo estadístico sobre datos conocidos.
RF-16	Should	El sistema debería calcular métricas adicionales específicas de Scrum cuando existan datos suficientes, como el cumplimiento del compromiso del sprint.	Prueba con datos completos de sprint.

RF-17	Could	El sistema podría calcular métricas adicionales específicas de Kanban cuando existan datos suficientes, como la eficiencia de flujo.	Prueba condicionada a la disponibilidad de datos Kanban.
-------	-------	--	--

3.6.4. Dashboard y visualización

ID	Prioridad	Descripción	Verificación
RF-19	Must	El sistema debe proporcionar al menos un dashboard principal con visualización de las métricas clave en un periodo seleccionable.	Revisión funcional del dashboard principal.
RF-20	Must	El sistema debe permitir filtrar dashboards y vistas analíticas por rango temporal y por atributos del trabajo importado.	Prueba funcional de filtros del dashboard.
RF-21	Should	El sistema debería permitir personalizar parcialmente el dashboard, seleccionando qué métricas o widgets se muestran.	Prueba funcional de personalización.
RF-22	Should	El sistema debería mostrar ayudas contextuales que expliquen la interpretación de las métricas y visualizaciones.	Revisión de interfaz y textos de ayuda.
RF-23	Could	El sistema podría proporcionar un modo de visualización simplificado para reuniones o presentaciones.	Revisión funcional del modo simplificado, si se implementa.

3.6.5. Alertas y logs

ID	Prioridad	Descripción	Verificación
RF-24	Must	El sistema debe permitir definir reglas de alerta simples, basadas en umbrales configurables sobre métricas calculadas, como WIP, Cycle Time, Lead Time o Velocity.	Prueba funcional con umbrales de ejemplo.
RF-25	Must	El sistema debe detectar y registrar las alertas activadas cuando se cumplan las condiciones configuradas.	Prueba de activación de alertas.
RF-26	Must	El sistema debe ofrecer una vista para consultar alertas activas e históricas, incluyendo su estado y momento de activación.	Revisión funcional de la vista de alertas.

RF-27	Must	El sistema debe registrar logs de eventos relevantes, incluyendo al menos sincronizaciones, errores de integración, errores de cálculo y eventos de autenticación.	Revisión de logs generados durante pruebas.
RF-28	Should	El sistema debería ofrecer una vista para consultar los logs del sistema con criterios básicos de filtrado.	Revisión funcional de la vista de logs, si se implementa.

3.6.6. Autenticación y control de acceso

ID	Prioridad	Descripción	Verificación
RF-29	Must	El sistema debe gestionar la autenticación exclusivamente mediante el mecanismo oficial de autenticación de Jira o Atlassian.	Prueba de autenticación con cuenta de Atlassian.
RF-30	Must	El sistema no debe permitir registro local de usuarios ni autenticación mediante credenciales propias ajenas a Jira.	Revisión funcional y de código.
RF-31	Must	El sistema debe proteger las operaciones de configuración, sincronización y consulta de información de acuerdo con la sesión autenticada a través de Jira.	Pruebas de autorización por sesión.

3.7. Requisitos no funcionales

Los requisitos no funcionales recogen propiedades de calidad, restricciones técnicas y criterios generales que debe cumplir el sistema.

ID	Prioridad	Descripción	Verificación
RNF-01	Must	El sistema debe ofrecer una interfaz clara y adaptable a dispositivos de escritorio y móviles, permitiendo consultar las métricas principales sin pérdida de información esencial.	Revisión funcional en distintos tamaños de pantalla.
RNF-02	Must	El sistema debe preservar la integridad y consistencia de los datos importados y calculados.	Pruebas de importación, cálculo y revisión de datos.
RNF-03	Must	El sistema debe persistir los datos necesarios para soportar análisis históricos y comparativos.	Revisión de almacenamiento y consulta histórica.
RNF-04	Must	El sistema debe registrar eventos relevantes de operación, errores de integración y fallos de procesamiento para facilitar trazabilidad y diagnóstico.	Revisión de logs durante pruebas funcionales.
RNF-05	Must	El sistema debe proteger credenciales, tokens y demás información sensible mediante mecanismos de almacenamiento y tratamiento seguro.	Revisión de código, configuración y tratamiento de secretos.
RNF-06	Must	El sistema debe minimizar el impacto de los límites de uso de servicios externos mediante estrategias de control de acceso, reducción de peticiones o almacenamiento temporal.	Pruebas de integración y revisión de llamadas externas.
RNF-07	Must	El sistema debe mantener tiempos de respuesta razonables para la consulta interactiva de dashboards en condiciones normales de uso.	Pruebas básicas de rendimiento.
RNF-08	Should	El sistema debería permitir ampliar el número de proyectos, periodos analizados y métricas sin degradación severa del servicio.	Pruebas de carga o revisión técnica de escalabilidad.
RNF-09	Must	El sistema debe exponer interfaces de comunicación internas y externas de forma consistente y documentada, especialmente en la comunicación entre frontend, backend y servicios externos.	Revisión de documentación técnica e interfaces.
RNF-10	Must	El sistema debe diseñarse para facilitar mantenimiento, prueba y evolución incremental.	Revisión de estructura del código y pruebas.

3.8. Requisitos de información

Los requisitos de información describen los datos que el sistema debe gestionar para poder ofrecer sus funcionalidades. En este proyecto incluyen tanto la información obtenida desde Jira Cloud como la información generada o configurada dentro del propio sistema, por ejemplo proyectos, incidencias, estados, métricas calculadas, filtros, alertas y parámetros de análisis.

Este tipo de requisito ayuda a identificar qué información relevante debe estar disponible para alcanzar los objetivos del sistema [14]. En el caso de este proyecto, resultan especialmente importantes porque las métricas ágiles dependen directamente de la calidad, disponibilidad y estructura de los datos procedentes de Jira.

ID	Prioridad	Descripción	Datos principales	Origen
RI-01	Must	El sistema debe gestionar información básica de las incidencias importadas desde Jira cuando estos datos estén disponibles.	Identificador, resumen, tipo, estado y prioridad.	Jira Cloud API
RI-02	Must	El sistema debe obtener y gestionar el historial de transiciones de estado de las incidencias para calcular métricas temporales.	Estado origen, estado destino, fecha y hora de transición, incidencia asociada.	Jira Cloud API
RI-03	Must	El sistema debe gestionar información sobre el esfuerzo asociado a las incidencias cuando sea necesario para calcular Velocity u otras métricas equivalentes.	Puntos de historia, estimación temporal o campo de esfuerzo configurado.	Jira Cloud API
RI-04	Must	El sistema debe gestionar información de sprint o periodo de trabajo cuando exista.	Nombre, fecha de inicio, fecha de fin y fechas relevantes.	Jira Cloud API
RI-05	Must	El sistema debe gestionar la relación entre incidencias, proyectos, tableros y equipos necesaria para segmentar los análisis.	Proyecto, tablero, sprint, equipo y relaciones con incidencias.	Jira Cloud API
RI-06	Should	El sistema debería conservar información de sincronización asociada a los datos importados.	Fecha y hora de actualización, origen de datos y ámbito sincronizado.	Sistema propio
RI-07	Must	El sistema debe incluir un conjunto mínimo de datos de prueba o demostración que permita validar su comportamiento en ausencia de conexión con fuentes reales.	Proyectos, incidencias, transiciones, sprints y métricas simuladas.	Sistema propio
RI-08	Must	El sistema debe gestionar información de configuración del análisis.	Filtros aplicados, estados de inicio y fin, rangos temporales, campos de esfuerzo y parámetros de cálculo.	Sistema propio

RI-09	Must	El sistema debe gestionar información sobre las métricas calculadas.	Tipo de métrica, valor obtenido, periodo analizado, proyecto asociado y filtros aplicados.	Sistema propio
-------	------	--	--	----------------

3.9. Casos de uso

A partir de los requisitos funcionales se identifican los principales casos de uso del sistema. Esta primera relación sirve como base para elaborar posteriormente el diagrama de casos de uso y las descripciones detalladas que se consideren necesarias. En una fase posterior, cada caso de uso podrá completarse con actores, precondiciones, flujo principal, flujos alternativos y postcondiciones.

ID	Caso de uso	Descripción
CU-01	Autenticarse mediante Atlassian	El usuario accede al sistema utilizando el mecanismo oficial de autenticación de Jira o Atlassian.
CU-02	Seleccionar ámbito de análisis	El usuario selecciona proyecto, tablero, sprint, rango temporal u otros filtros disponibles.
CU-03	Sincronizar datos de Jira	El usuario ejecuta una sincronización bajo demanda para importar o actualizar datos desde Jira Cloud.
CU-04	Configurar parámetros de métricas	El usuario define estados de inicio y fin, campos de esfuerzo y otros parámetros necesarios para calcular métricas.
CU-05	Consultar dashboard principal	El usuario visualiza las métricas clave del sistema en un dashboard principal.
CU-06	Analizar métricas de flujo	El usuario consulta métricas como Lead Time, Cycle Time, WIP y Throughput o Velocity según el contexto.
CU-07	Configurar reglas de alerta	El usuario define umbrales simples para detectar situaciones relevantes en las métricas calculadas.
CU-08	Consultar alertas	El usuario revisa alertas activas e históricas generadas por el sistema.
CU-09	Consultar logs del sistema	El usuario o perfil técnico consulta eventos relevantes del sistema para diagnóstico y seguimiento.

3.10. Trazabilidad entre requisitos y casos de uso

La trazabilidad permite comprobar que los casos de uso identificados cubren los requisitos funcionales principales. En esta fase inicial se plantea una matriz preliminar, que podrá ajustarse cuando se definan con mayor detalle los casos de uso y los diagramas asociados.

Tabla 3.10: Matriz de trazabilidad entre requisitos funcionales y casos de uso

Req.	CU-01	CU-02	CU-03	CU-04	CU-05	CU-06	CU-07	CU-08	CU-09
RF-01			X						
RF-02		X							
RF-03			X						
RF-04			X						
RF-05			X						
RF-06			X						
RF-07		X		X					
RF-08		X		X					
RF-09					X	X			
RF-10						X			
RF-11			X						
RF-12						X			
RF-13						X			
RF-14						X			
RF-15						X			
RF-16						X			
RF-17						X			
RF-18				X					
RF-19					X				
RF-20		X			X				
RF-21					X				
RF-22					X				
RF-23					X				
RF-24							X		
RF-25								X	
RF-26								X	
RF-27									X
RF-28									X
RF-29	X								
RF-30	X								
RF-31	X								

La matriz anterior no sustituye a la descripción detallada de los casos de uso. Su función es validar de forma visual que los requisitos funcionales quedan cubiertos por, al menos, un caso de uso identificado.

Bibliografía

- [1] Ken Schwaber y Jeff Sutherland. *The Scrum Guide*. 2020. URL: <https://scrumguides.org/scrum-guide.html> (visitado 10-08-2026).
- [2] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010. ISBN: 9780984521401.
- [3] Digital.ai. *17th State of Agile Report*. 2024. URL: <https://digital.ai/press-releases/17th-state-of-agile-report-71-use-agile-in-their-sdlc-small-organizations-report-strong-business-benefits-medium-and-larger-sized-companies-continue-to-experience-barriers-in-successfully-scaling-a/> (visitado 10-08-2026).
- [4] Atlassian. *Jira*. URL: <https://www.atlassian.com/software/jira> (visitado 10-08-2026).
- [5] DORA. *DORA's software delivery performance metrics*. 2026. URL: <https://dora.dev/guides/dora-metrics/> (visitado 10-08-2026).
- [6] Atlassian. *Jira Pricing*. URL: <https://www.atlassian.com/software/jira/pricing> (visitado 10-08-2026).
- [7] LinearB. *Jira Metrics and Dashboards for Engineering Teams*. URL: <https://linearb.io/integrations/jira> (visitado 10-08-2026).
- [8] Swarmia. *Integrations*. URL: <https://www.swarmia.com/integrations/> (visitado 10-08-2026).
- [9] Atlassian. *Create a board*. URL: <https://support.atlassian.com/trello/docs/creating-a-new-board/> (visitado 10-08-2026).
- [10] Taiga. *Taiga: Your open source agile project management software*. URL: <https://taiga.io/> (visitado 10-08-2026).
- [11] Microsoft. *What is Azure DevOps?* URL: <https://learn.microsoft.com/en-us/azure/devops/user-guide/what-is-azure-devops> (visitado 10-08-2026).
- [12] IEEE Computer Society. *IEEE Recommended Practice for Software Requirements Specifications*. IEEE Std 830-1998. Reaffirmed 2009. IEEE. 1998. URL: https://bohr.wlu.ca/cp317/notes/IEEE_830.pdf (visitado 14-08-2026).
- [13] Agile Business Consortium. *What is MoSCoW Prioritization?* URL: <https://www.agilebusiness.org/resource/what-is-moscow-prioritization/> (visitado 14-08-2026).

- [14] Amador Durán y Beatriz Bernárdez. *Metodología para la Elicitación de Requisitos de Sistemas Software*. Informe técnico LSI-2000-10. Departamento de Lenguajes y Sistemas Informáticos, Universidad de Sevilla, 2000. URL: https://www.researchgate.net/publication/266277994_Metodologia_para_la_Elicitacion_de_Requisitos_de_Sistemas_Software (visitado 14-08-2026).

Apéndice A

Resultados detallados de la experimentación

En este apéndice se incluyen una serie de tablas que se ha considerado que, incluidas dentro del capítulo de experimentación, dificultaban su lectura y la comprensión de la estructura experimental.

Estas tablas describen...

- BuzzFeed [1]: tablas A.1 a A.1.
- Politifact: ...

Ejemplo de estilo para poner dos figuras en paralelo:

Ejemplo para casos de uso:

Referencias

- [1] BuzzFeedNews. *Fact-Checking Facebook Politics Pages*. 2016. URL: <https://github.com/BuzzFeedNews/2016-10-facebook-fact-check/> (visitado 09-06-2020).

Tabla A.1: Resultados de regresión logística en Buzzfeed

Fichero	Balanceo	% Clase mayoritaria	Acc. (%) entrena- miento	Acc. (%) test
fase_1_y _2_data.cs v	Sin balanceo	84	85.7 ± 0.6	84.0 ± 2.7
fase_1_y _2_data.cs v	Upsample	50 en train y 84 en test	79.8 ± 1.8	72.7 ± 4
fase_1_y _2_data.cs v	Downsample	50 en train y 84 en test	79.0 ± 3.3	71.1 ± 3.5
fase3_da ta_1_Gra ms.csv	Sin balanceo	84	98.5 ± 1.8	86.4 ± 4.1
fase3_da ta_1_Gra ms.csv	Upsample	50 en train y 84 en test	99.5 ± 0.1	86.3 ± 3.4
fase3_data _1_Grams .csv	Downsample	50 en train y 84 en test	95.4 ± 1.03	74.7 ± 6.4
fase3_data _2_Grams .csv	Sin balanceo	84	99.2 ± 0.2	85.0 ± 5.2
fase3_data _2_Grams .csv	Upsample	50 en train y 84 en test	99.5 ± 0.001	84.7 ± 4.3
fase3_da ta_2_Gra ms.csv	Downsample	50 en train y 84 en test	94.4 ± 7.3	77.6 ± 2.5
fase3_data _3_Grams .csv	Sin balanceo	84	88.7 ± 7.2	84.3 ± 1.7
fase3_data _3_Grams .csv	Upsample	50 en train y 84 en test	99.2 ± 0.4	81.8 ± 3.1
fase3_data _3_Grams .csv	Downsample	50 en train y 84 en test	93.3 ± 8.3	77.6 ± 5.8

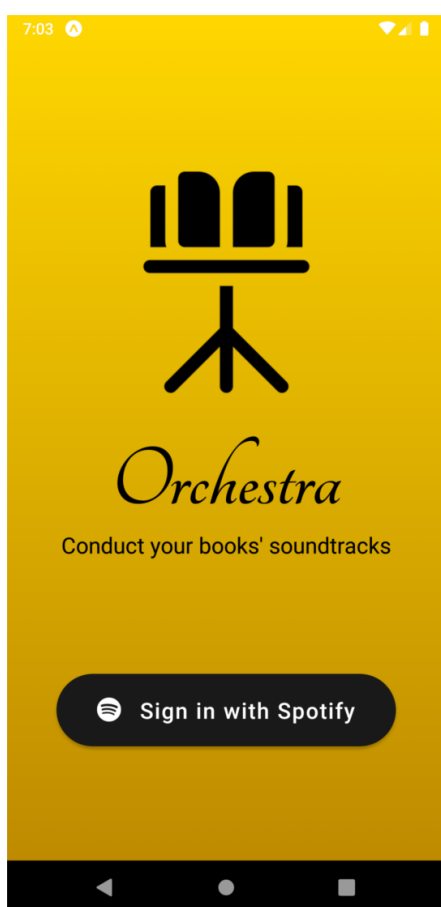


Figura A.1: Pantalla de acceso.

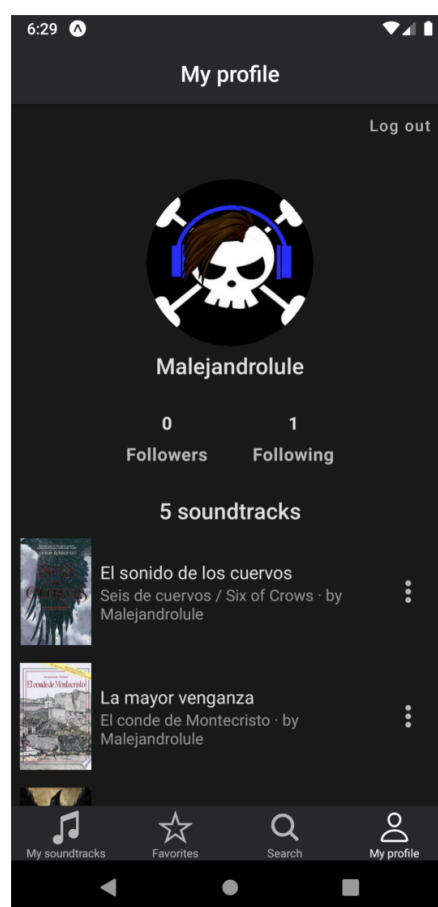


Figura A.2: Pantalla de perfil de usuario.

Caso de Uso	Nombre del caso de uso
Actores	Usuario no registrado.
Descripción	El objetivo del caso de uso es que cualquier usuario pueda registrarse para hacer uso de la aplicación.
Precondición	El usuario no debe estar previamente registrado en la aplicación.
Flujo principal	1. El Caso de Uso comienza cuando el usuario selecciona la opción de registro. 2. La Aplicación solicita al usuario un email y una contraseña. 3. El usuario introduce el email. 4. El usuario introduce la contraseña. 5. La aplicación valida que el email y la contraseña es correcto. 6. Mientras la contraseña o email no sean correctos. a) La aplicación solicita al usuario un email. b) El usuario introduce el email. c) La aplicación solicita al usuario una contraseña. d) El usuario introduce la contraseña. e) La aplicación valida los datos son correctos. 7. El usuario selecciona la opción de finalizar registro 8. La aplicación informa al usuario que se ha registrado correctamente.
Postcondición	El usuario se ha registrado aplicación.
Flujo Alternativo	Cancelar: el usuario cancela el registro y vuelve a la página de Inicio.

Tabla A.2: CU01: Registro en la aplicación

Apéndice B

Manual de usuario

Aquí el manual de usuario. También podría ponerse en un documento a parte. Hay quien prefiere que sean documentos separados porque en la realidad deben serlo ya que el destinatario es distinto. Hay quien prefiere que estén como anexos porque así en un documento único está todo.

Apéndice C

Manual de código fuente

C.1. Contenidos del manual de código fuente

- Introducción del documento, explicando: a que proyecto corresponde, que lenguajes de programación, entornos, o librerías se han utilizado, y todo aquello que tenga relación con como se ha creado y organizado el código fuente. (obligatorio)
- Instrucciones de compilación, empaquetado, actualización... (si proceden)
- Instrucciones de instalación y desinstalación (si procede aquí en vez de en el manual de usuario)
- Descripción de la organización del código en archivos y como se corresponde con la organización modular explicada en la memoria técnica. (obligatorio)
- Sí ayuda a la comprensión de la organización del código y se considera útil, documentación generada por sistemas automáticos de documentación, o breve explicación y enlace a la web (ej.) donde se puede encontrar esta documentación. (si procede)
- Todos los ficheros con el código fuente de la aplicación (ej. código fuente .py, código fuente .c, cabeceras .h, Makefile, configure, ficheros de creación de bases de datos .sql, páginas web .html, código fuente .php, ficheros de carga de datos básicos, scripts del sistema operativo .sh....) (obligatorio, puede ser en repositorio accesible)

En vez de preparar todo el código con estilo de impresión, es adecuado incluir el código en un repositorio donde el tribunal pueda consultarlo indicandolo en este manual. En ese caso, ya no es necesario poner el código en este manual pero sí las explicaciones de su estructura. Ejemplo:

... todo el código aquí descrito se puede encontrar en el siguiente repositorio: <https://gitlab.com/angelpavon97/fake-news-detector/>

C.2. Código fuente de otros autores

No se debe incluir código fuente que no haya sido desarrollado por el autor para el proyecto.

- Si hay ficheros completos tomados de otras fuentes que son necesarios para el proyecto, estos deberán ser referenciados y explicar como se integran en el proyecto.
- Si se han realizado modificaciones a código de otras fuentes, cuando se incluyan estos ficheros modificados, se deberá explicar claramente su origen y el alcance de las modificaciones realizadas. Todo esto debe quedar muy claro, de forma que no dé lugar a duda sobre la intención del autor al incluirlo en el manual.
- Nota: en cualquier caso, el autor debe recordar que debe cumplir con toda la legislación sobre derechos de propiedad intelectual.

C.3. ¿Qué va a mirar el tribunal en tú código?

Puede ver que el código esté desarrollado de forma adecuada. Por ejemplo, en una ocasión vi una secuencia de una centena de instrucciones `if` encadenadas que claramente deberían haber sido sustituidas por un vector de datos y el acceso al mismo. Otra cosa que se puede es si hay valores insertados en el código que deberían ser configuraciones (*hard-coded*). Si has hecho un código con un mínimo de cuidado, no es algo que deba preocuparte.

También puede mirar que el estilo de código sea consistente (igual a lo largo de todo el código) y adecuado (nombres de variables descriptivos, no ofuscado...). Además, que se incluyen comentarios adecuados, que ayuden entender rápidamente el código y trabajar con él, a la vez que se siguen las reglas de Ottinger:

1. Comments are for things that cannot be expressed in code.
2. Comments which restate code must be deleted.
3. If the comment says what the code could say, then the code must change to make the comment redundant.

C.4. Estilo del texto de los ficheros de código fuente

Es importante que para incluir el código fuente se sigan algunas normas básicas. Se debe recordar siempre que la finalidad es facilitar el estudio y comprensión del código. Por tanto, éste deberá aparecer de la forma más parecida posible a como aparece en el editor y es mejor tener la mayor cantidad posible de código en cada página para reducir al máximo el tener que andar pasando páginas hacia atrás y hacia delante. No debe en ningún caso usar un estilo con el objeto de engordar el código y que aparente mayor volumen de trabajo (en muchos casos esto consigue justo lo contrario).

El estilo para el código fuente debería:

- Tener espaciado entre líneas y párrafos a cero.
- Fuentes de tamaño fijo o monoespacio (todos los caracteres ocupan lo mismo). Esto hace que los indentados para indicar los bloques de código se vean bien y que todo el código se alinee igual que en el editor.
- Un tamaño de fuente pequeño, para evitar, o al menos reducir, que las líneas no quepan y se pasen a la siguiente. Como el código no va a ser leído de forma continuada, no importa que la letra tenga un tamaño más reducido que en el resto de la documentación, pero debe ser un tamaño legible.

También es posible jugar con el formato (reducir márgenes, ponerlo a doble columna) para que ocupe menos páginas. A continuación se mostrarán ejemplos de código estilo.

Ejemplo de cita al lenguaje de programación:

... se ha realizado con el lenguaje de programación Python [1] ...

Ejemplo de un listado de directorio:

```
BaseDeDatosEjemplo
├── data
│   ├── True
│   │   ├── 1.txt
│   │   ├── 2.txt
│   │   └── 3.txt
│   └── False
│       ├── 1.txt
│       ├── 2.txt
│       └── 3.txt
├── BaseDeDatosEjemplo_1grams.csv
├── BaseDeDatosEjemplo_2grams.csv
├── BaseDeDatosEjemplo_3grams.csv
├── BaseDeDatosEjemplo_reduced_1grams.csv
├── BaseDeDatosEjemplo_reduced_2grams.csv
├── BaseDeDatosEjemplo_reduced_3grams.csv
├── fase1_data.csv
├── fase2_data.csv
├── fase3_data_1_Grams.csv
├── fase3_data_2_Grams.csv
├── fase3_data_3_Grams.csv
└── fase_1_y_2_data.csv
```

Ejemplo de listado código fuente Python:

```

1 import csv
2 import requests
3 import urllib.request
4 from bs4 import BeautifulSoup
5
6 def scraping_abc(url):
7     """
8     Comentario de la función
9     """
10    try:
11        page = urllib.request.urlopen(url)
12    except:
13        print("An error occurred.")
14        return 'error', 'error'
15
16    soup = BeautifulSoup(page, 'html.parser')
17
18    # Buscamos todos los parrafos (contenido)
19    content_lis = soup.find_all('p')
20
21    # Buscamos el titulo
22    header = soup.find('header', attrs={'class': 'Article__Header'})
23    title = header.find('h1').getText()
24
25    # Imprimimos
26    print('Title: ' + title)
27    content = ''
28    for p in content_lis:
29        content = content + p.getText()
30    print(content)
31
32    return title, content

```

Y un HTML:

```

1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5     <meta charset="UTF-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <meta http-equiv="X-UA-Compatible" content="ie=edge">
8     <!-- Bootstrap 4 -->
9     <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/
10         css/bootstrap.min.css"
11         integrity="sha384-Vkoo8x4CGsO3+Hhvx8T/
12         Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh" crossorigin="anonymous">
13
14     <!-- Custom css -->
15     <link rel="stylesheet" href="{url_for('static', filename='css/main.css')}">
16
17     <script src="{url_for('static', filename='js/app.js')}"></script>
18
19     <title>Fake News Detector</title>
20 </head>
21
22 <body>
23     <nav class="navbar navbar-expand-lg navbar-light bg-light">
24         <a class="navbar-brand" href="/">Fake News Detector</a>
25         <button class="navbar-toggler" type="button" data-toggle="collapse"
26             data-target="#navbarText"
27             aria-controls="navbarText" aria-expanded="false" aria-label="Toggle
28                 navigation">
29             <span class="navbar-toggler-icon"></span>
30         </button>
31         <div class="collapse navbar-collapse" id="navbarText">

```



```
28         <ul class="navbar-nav ml-auto">
29             <li id="nav-home" class="nav-item">
30                 <a class="nav-link" href="/">Home</a>
31             </li>
32             <li id="nav-project" class="nav-item">
33                 <a class="nav-link" href="{{url_for('project')}}">The Project</a>
34             </li>
35             <li id="nav-classifier" class="nav-item">
36                 <a class="nav-link" href="{{url_for('classifier')}}">Classifier
37                     </a>
38             </li>
39         </ul>
40     </div>
41 </nav>
42
43     {% block content %}
44     {% endblock %}
45 </body>
46
47 </html>
```

Referencias

- [1] Python Software Foundation. *Python (Versión 3.7) [Software]*. 2018. URL: <http://www.python.org> (visitado 02-07-2020).

